

CommonsMesh — NGI Zero Commons Fund Application Attachment

Supplementary Technical Proposal and Milestone Plan

Submitted as an attachment to the NGI Zero Commons Fund application.

Part A: Strategic Alignment with the Next Generation Internet Vision

Architectural field research conducted in the informal settlements of Port Moresby, Papua New Guinea, provided firsthand observation of how communities survive on the margins. These communities do not rely on state infrastructure; instead, they depend on dense, informal networks of mutual aid. They share tools, labor, and information. However, coordinating this sharing is incredibly difficult, particularly when internet access is intermittent and individuals are time-poor.

Currently, these communities are forced to utilize corporate platforms like WhatsApp or Facebook Groups for organization. This presents a mismatch for two primary reasons. First, chat applications are designed for conversation, not coordination, resulting in critical needs being easily buried. Second, these platforms extract data, trapping vulnerable populations in surveillance capitalism.

CommonsMesh is designed as a technical alternative that aligns directly with the Next Generation Internet's vision of a human-centric, trustworthy web. It provides a local-first, privacy-preserving coordination layer. Built on libp2p, it eliminates the need for a central server, ensuring data remains exclusively on the users' devices.

By transitioning from a “social feed” model to a “structured community graph” model, CommonsMesh transforms chaotic group chats into actionable mutual aid. Furthermore, by embedding a lightweight LLM directly on the mobile device to assist in matching needs and resources, it delivers the benefits of AI without transmitting community data to a cloud provider. This represents the type of open, decentralized public good that the NGI initiative aims to support.

Part B: Detailed Technical Architecture

The optimization techniques developed over five years of writing highly optimized code for smart glasses—edge devices with severe constraints on battery, memory, and connectivity—are being directly applied to the development of CommonsMesh.

B.1 Protocol Layer

The core of CommonsMesh resides in its protocol, where every community action is treated as a cryptographically signed event envelope.

When a user interacts with the application, an event is generated (e.g., `graph.delta` to declare a need, or `project.motion` to propose a group action). This event is signed using an Ed25519 key generated on the device. Canonical JSON serialization and a SHA-256 hash are utilized to ensure the payload cannot be tampered with. Each event links to the preceding one via a `chain_hash`, creating a tamper-evident, append-only log for every user.

To mitigate Sybil attacks (spamming the network with fake accounts) within a serverless environment, a multi-layered validator pipeline has been designed. It verifies temporal windows (rejecting events older than 5 minutes to prevent replay attacks) and enforces a local trust-scoring system. High-impact actions require a minimum trust score, which is accumulated through social proofs (`trust.attest` events) from existing community members.

B.2 Network Layer (libp2p)

The network layer is constructed on libp2p. The node factory is configured to manage the complexities of mobile networks: utilizing TCP and WebSockets for transport, mDNS for discovering peers on the same local network, and GossipSub for broadcasting events.

A significant challenge in mobile P2P is state synchronization when devices frequently disconnect and reconnect. An incremental sync protocol using Bloom filters is being implemented to address this. When two devices connect, they exchange small Bloom filters representing their known event hashes. They calculate the difference and

request only the missing segments. This approach maintains extremely low bandwidth usage, a critical requirement for users on prepaid data plans.

B.3 Local Graph Database

All data is stored locally utilizing SQLite (via `expo-sqlite` within React Native). The schema features an append-only `events` table that stores the raw cryptographic envelopes. From this table, the application derives `graph_nodes` and `graph_edges` tables, representing the current state of the community (e.g., identifying needs and project participants).

This local graph serves as the data source for both the UI queries and the on-device AI. Because the data is local, the application functions seamlessly offline, synchronizing state changes upon the next peer connection.

B.4 On-Device LLM Integration

The integration of `llama.cpp` into the React Native application using `llama.rn` enables the local execution of GGUF-format models (such as Qwen2.5 1B). This leverages the edge computing optimization techniques previously mentioned.

To conserve battery life, the matching engine operates in two stages. The first stage employs a deterministic, rule-based engine that searches for exact keyword matches within the local SQLite graph. If a match is identified, the process concludes.

The second stage activates the LLM only when the device is idle or charging. It provides the LLM with a JSON summary of unmatched needs and resources. The prompt instructs the model to identify semantic matches (e.g., recognizing that an offer of “truck space” aligns with a need for “moving boxes”) or to detect aggregate patterns (e.g., observing that five individuals require the same building material, prompting a bulk purchase motion). The LLM outputs structured JSON, which the application translates into actionable UI elements.

Part C: Milestone Plan

The project is structured as a 9-month development cycle. It is being developed independently, with each milestone resulting in a concrete, open-source deliverable.

Milestone	Duration	Deliverables	Verification
M1: Protocol & Security	Months 1–2	Finalized event schema, the validator pipeline (anti-Sybil, replay protection), and the capability token system.	Code pushed to GitHub; test suite passing.
M2: P2P Network Sync	Months 3–4	libp2p node setup for mobile, Bloom filter sync implementation.	Code pushed to GitHub; demo video showing two nodes syncing.
M3: Local AI Integration	Months 5–6	llama.rn integrated, two-stage matching engine built, prompts optimized for 1B models.	Code pushed to GitHub; benchmark report on inference time/battery usage.
M4: Mobile App Build	Months 7–8	React Native (Expo) app completed, UI wired to the local graph DB.	Code pushed to GitHub; end-to-end test video on physical Android/iOS devices.
M5: Pilot & Docs	Month 9	Run a pilot with a local mutual aid group, write the protocol specs, prep for security audit.	Pilot report and technical documentation published.

Part D: Sustainability and Long-Term Impact

All components are being released under the AGPL-3.0 license, ensuring the codebase remains a digital commons. Any commercial entity utilizing the protocol is required to contribute their improvements back to the community.

The local-first and serverless architecture eliminates ongoing cloud hosting costs. The community sustains the network simply by running the application on their devices.

The long-term objective is for the CommonsMesh protocol to become a standard building block. Similar to how ActivityPub standardized federated social media, this protocol aims to standardize decentralized community coordination. Widespread adoption of the event schema could facilitate interoperable mutual aid networks that do not rely on any single application or platform.

Part E: Relation to Existing NGI Zero Projects

The project heavily leverages the libp2p ecosystem, which has previously received NGI support. While projects like Secure Scuttlebutt focus on federated social feeds, CommonsMesh is strictly focused on structured coordination and state changes. It occupies a distinct niche: it is designed for actionable coordination rather than conversation.

Collaboration with other NGI Zero grantees is welcomed, particularly those working on local-first data sync or decentralized identity, given the significant overlap with the goals of CommonsMesh.

Part F: Open Source Commitment

A commitment is made to release all software, documentation, and protocol specifications produced under this grant under the **AGPL-3.0-only** license.

The repository is publicly available at: <https://github.com/e982happy/CommonsMesh>

The repository will be maintained, the code thoroughly documented, and all work funded by NGI Zero will be easily accessible and reusable by the broader open-source community.